University of Cape Town
Department of Computer Science

Computer Science CSC3003S
November Examination, 2017

Read through the entire examination paper before you start writing so that you can answer the easy questions first. The relative importance of a question is indicated by the marks allocated to it. Please write legibly.

This paper consists of Three Sections:

Section A: Advanced Software Design 1

Section B: Advanced Software Design 2

Section C: Theory of Algorithms/Theory of Computation

Total Marks: 90	Approximate marks per question are shown in brackets.
Time: 3 hours + 10 minutes reading	The use of calculators is permitted.

Please write your answers in three separate books. Use one book for Advanced Software Design 1, one book for Advanced Software Design 2, and one book for Theory of Algorithms/Theory of Computation. **Label each of the three books as ASD1, ASD2 and ToA/ToC**.

Your attention is drawn to the instructions on your answer book on the front and the back. In particular, "**EVERY CANDIDATE MUST enter the book number and the number of each question answered (in the order in which it has been answered)**" on the table provided on the front cover of the first book.

Please **do this for the front cover of the different books used for each section.**

This is needed so that we can allocate marks and not miss questions. **A list of sub-questions is *not* required.**

Section A. [24]

Question 1. **[8]**

a. You are selecting a team for a new project. Explain the purpose of the following roles in small software development teams: implementer, integrator, project manager, software architect, and test manager. [5]

b. Name three factors or information sources you should take into consideration when choosing people for your team [3]

Question 2. **[9]**

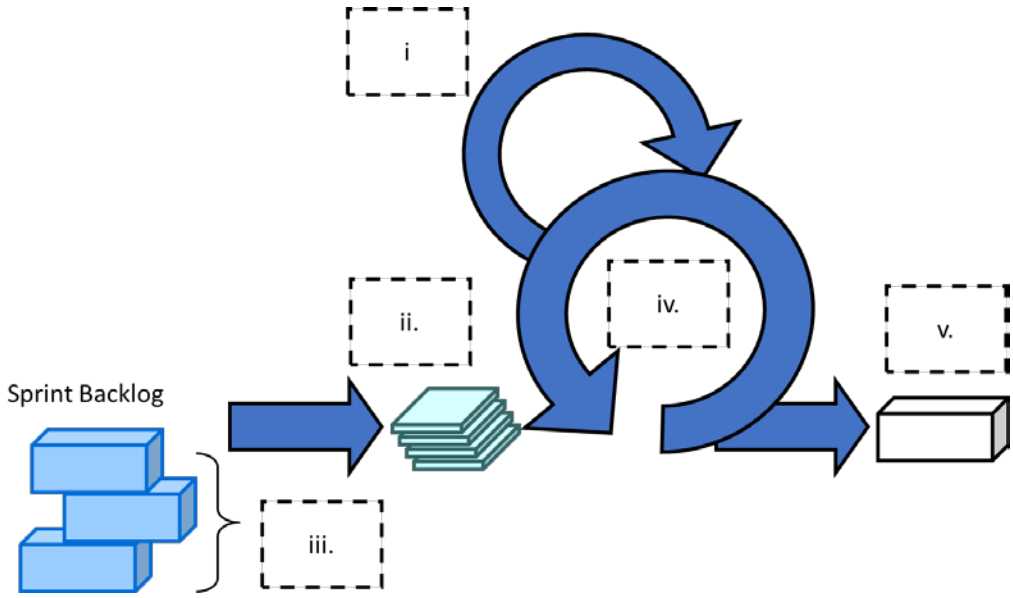
a. You Ain't Gonna Need It (YAGNI) is a cry to prevent speculative development. Explain what speculative development is and why it is bad for software development. [3]

b. How do iterative processes accommodate risk better than the waterfall approach? [6]

Question 3. **[7]**

a. Please provide the missing labels for the following diagram, and indicate lengths of time or roles responsible for each. [5]

b. What is meant by "velocity" in a Scrum Burndown chart and why is it important to the Scrum team? [2]



Section B. [26]

Question 4. Architectural Thinking [8]

a. Provide a short definition of Software Architecture that captures the essential points of what it is about. [2]

b. In the "4+1" view of architectures the 4 components are the [2]

1. Logical View
2. Development View
3. Process View, and
4. Physical View.

What does the "+1" in the above refer to? What purpose does it serve?

c. Pick two of the components listed above in question b and describe what they represent and what purpose they serve. [4]

Question 5. Software [10]

a. We characterized the advanced software "design" course in contrast to the "analysis" course that was studied before. *Briefly* compare and contrast these two terms. What is "analysis" about? What is "design" about? [4]

b. Which Gang of Four (GOF) design pattern is applied in the code snippet below? [4]

```
public class PrintSpooler {
    private static final PrintSpooler
        INSTANCE = new PrintSpooler();
    private PrintSpooler() {}
    public static PrintSpooler getInstance() {
        return INSTANCE;
    }
}
```

It is one of the following: PrintSpooler, Spooler, Singleton or Factory. Explain what the pattern is and why it is applicable here.

c. You believe that people should own their own software, code and all, and companies shouldn't have the right to grab previously open software in packages and so force people to buy products that started out as truly free. What type of open source license would you want to use? Justify your choice. [2]

Question 6. Some Patterns inside Model-View-Controller [8]

You have met the pattern *model-view-controller* in many guises in a number of courses. Here we want you to analyse it in terms of component patterns that make it up or get used inside it. The patterns you need to consider in this regard are: *Observer*, *Strategy*, *Composite* and *Adapter*.

Hint: to show you what is meant: we can say the Adapter pattern can be used to adapt a new model to an existing view and controller.

Section C. [40]

Question 7. [11]

a) How would the product of 2 integers ***m*** and ***n*** be computed using each technique below? You need just 1 line per answer.

   a. Brute force

   b. Decrease by a constant

   c. Decrease by a constant factor

   d. Divide and conquer [2]

b) Above we considered just 4 techniques. Name **4 other** problem solving techniques we studied, and briefly explain each of them. [4]

c) Choose the right option from each of the following so as to make that statement true:

   a. $\Theta(n + \log n)$ **equals / is not equal to** $\Theta(n)$

   b. $\Theta(\log^2 n)$ **equals / is not equal to** $\Theta(\log n)$

   c. $O(n \log n)$ **equals / is not equal to** $O(n)$

   d. if $x \in \Theta(n \log n)$ then $x \in$ **(log n) / O (log n)**

   e. if $x \in O(n \log n)$ then $x \in$ **O (log n) / O (2^n)**

   f. if $x \in (n \log n)$ then $x \in$ **(log n) / (2^n)** [3]

d) What would the values of ***a, b*** and ***d*** be if we used the Master Theorem to find the efficiency of the following function? **Just give the 3 values, showing clearly which is which**.

waat (x) { if x = 0 return a[0]; a[x] = waat(x/3 + 1) + waat(x/3 – 1); return a[x] }

Master Theorem: If $T(n) = aT(n/b) + f(n)$ then $T(n)$ $O(n^d)$ if $a < b^d$

$T(n)$ $O(n^d \log n)$ if $a = b^d$

T(n) $O(n^{\log_b a})$ if $a > b^d$ [2]

Question 8. **[13]**

a) Choose **any 3 (three)** of the problems below and say how could you most efficiently solve it. Give 1 or 2 lines per answer.

1. You have to find the one fake coin in a huge bag of coins. You have only a balance scale (it has 2 weighing cups; the cup with the heavier load of coins will sink lower than the cup with the ligher load). You know that the fake coin is lighter than the others. Brute force would take too long.
2. You have a huge file of names and are searching for your own name using a binary search, but it is taking too long.
3. Aliens must be moved from one planet to another using a spaceship that can only fit 3 aliens in it at a time, at least 1 of whom must be the pilot; there are many complicated constraints as to which aliens cannot be left alone together without a fight erupting. Brute force would take too long.
4. You must find the K'th largest value in a huge file of numbers. The numbers are not ordered.
5. You have a huge file of applicant data, each of which has applicant details including their grading as A, B, C, D or E. You must sort them in order of grading i.e. so all the A's are first, then all the B's, etc. Brute force would take too long.

[3]

b) Consider the alphabet ABCDE, where A occurs 15% of the time, B occurs 20% of the time, C occurs 20% of the time, D occurs 10% of the time and E occurs 35% of the time. What encoding would Huffman's Algorithm produce to compress strings from this alphabet? Show your working. [4]

c) Show by means of a small worked example of your own choice how **any 1 (one)** of the following algorithms works – just show the initialised array, and then the array obtained after the first iteration of the algorithm. This is enough to show how it works.

1. Warshall's algorithm for finding transitive closure. Your example should have about 4 nodes and about 4 edges.

OR

2. The knapsack problem. You should have about 4 items to choose from, and the knapsack should *not* have enough capacity to fit all 4 items in it. State whether you are using the memory function approach or not (no marks deducted if you don't use it).

OR

3. Computing binomial coefficients / finding the number of combinations (subsets) of size K chosen from a set of size N / computing Pascal's triangle. All 3 of these use the same recurrence relation: $F(n,k) = F(n-1,k-1) + F(n-1,k)$ for $n > 0$ and $k > 0$, with $F(n,0) = F(n,n) = 1$.

[4]

d) Given an MxN map, where you start at position (1,1), the top left corner, you want to know how many routes there are to get to (M,N), the bottom right corner of the map. The only moves you can make are to go down or to the right, and there are some grid positions – i.e. some cells (R,C) – that you cannot move into because of an obstacle there.
Example: if the map is 4x4 and there is only 1 obstacle in position (2,3), then the number of routes is 11.
Give an algorithm to solve this problem. Remember any point on the map you only be reached by coming either from the left or else from above. [2]

Question 9. **[6]**

A collection of gems is being sold. To prevent a single buyer from purchasing them all, the cost increases greater than linearly, with the number of gems bought. The total cost to buy K gems is f(K), where f(K) is the sum of j * (K/j) for all integers $0 < j < K$.

(Note K/j here uses integer division, so the result has no fractional part. For example: 8/3 would equal 2.)

As an example, to buy 5 gems, the cost, f(5), would be:

1 * (5/1) + 2 * (5/2) + 3 * (5/3) + 4 * (5/4) = 1*5 + 2*2 + 3*1 + 4*1 = 16

Problem: find the largest number of gems that can be bought for a given amount of money, i.e. given an amount of money, N, find the greatest value of K for which $f(K) \leq N$. Note that the function f(K) is strictly increasing, and that K is guaranteed to be at most 1 000 000.

a) Outline a brute force algorithm (in English or pseudocode) to solve this problem. [2]

b) Now give pseudocode for a more efficient algorithm to solve this problem. [3]

c) Give the efficiency of each of your 2 algorithms above. [1]

Question 10. **[5]**

Prove, using the formal definitions of Deterministic Finite Automata and Non-deterministic Finite Automata, that for every Deterministic Finite Automaton there is a Non-deterministic Finite Automaton that accepts exactly the same language.

Question 11. **[5]**

Suppose we want to prove that for any regular language A, A* is also regular. We know that if A is regular, then there is a Non-Deterministic Finite Automaton N such that N accepts A. Now, from N we construct a new Non-Deterministic Automaton N' as follows: for every accept state s of N, add an epsilon-transition from s to the start state of N (if there isn't one already). Does N' accept A*? If it does, explain why in detail. If it does not, explain why not, indicate what language it accepts, and explain how to modify N' so that it accepts A*.